

Systematic Design of RAG-based LLM Inference Systems: A Survey

Abstract

Retrieval-augmented generation (RAG) has become a practical way to improve large language model (LLM) responses with external knowledge, but its deployment cost is increasingly shaped by systems concerns rather than model design alone. Modern RAG serving pipelines must coordinate query encoding, vector retrieval, reranking, prompt construction, and autoregressive decoding under tight latency, memory, and cost constraints. The literature collected in this repository shows that recent progress is converging on a system-centric view of RAG inference, where GPU acceleration, cache reuse, pipeline scheduling, external-memory support, and application-specific optimization are treated as jointly designed components. This survey organizes that literature into a unified taxonomy of RAG inference system design. We first summarize the end-to-end RAG pipeline and the main bottlenecks that arise in production environments. We then review five major design axes: GPU-native acceleration, data reuse and cache-centric optimization, pipeline parallelism and scheduling, external-memory and storage-aware acceleration, and application-driven RAG systems. Across these axes, we highlight recurring trade-offs among latency, throughput, memory capacity, quality, freshness, and hardware specialization. We also provide a comparison table, synthesize major research trends, and outline open challenges for future work on efficient, scalable, and quality-aware RAG inference systems.

1. Introduction

Retrieval-augmented generation has become a core architecture for knowledge-intensive LLM applications because it separates parametric generation from non-parametric knowledge storage. Instead of depending only on model weights, a RAG pipeline retrieves relevant external context, injects it into the prompt, and then decodes a response conditioned on both the query and the retrieved evidence. This design can improve factuality, freshness, domain adaptation, and controllability. However, as RAG moves from research prototypes to deployed services, the main bottleneck is no longer just retrieval quality. It is the efficiency of the whole inference system.

In a production setting, RAG serving is a multi-stage pipeline with heterogeneous resource demands. Query encoding and vector search are often memory-bandwidth dominated. Reranking and prompt assembly introduce additional compute and data movement overheads. LLM decoding stresses GPU memory because of large model states and expanding key-value (KV) caches. The final end-to-end performance depends on how these stages are coordinated rather than on any single stage in isolation. For this reason, recent work increasingly treats RAG as a systems problem involving co-design across retrieval algorithms, hardware hierarchy, caching, scheduling, and memory management.

The paper list in this repository reflects that shift clearly. Recent systems such as RAGO and Hermes frame retrieval-augmented generation as an end-to-end serving problem. Works such as Cache-Craft, CacheBlend, MemoRAG, HyperRAG, CacheFocus, and METIS elevate reuse and adaptation from minor optimizations to first-class design principles. PipeRAG, RAGDoll, ELERAG, Patchwork, AquaPipe, and DGRAG show that overlapping retrieval and generation can be as important as optimizing either stage individually. A large body of work on SmartSSD platforms, computational storage, second-tier memory, and disk-native ANN indexing further demonstrates that GPU memory is insufficient for large-scale deployments.

This survey provides a system-centric view of the RAG inference landscape. Our contributions are threefold:

- We organize recent literature into a taxonomy of RAG inference system designs.
- We compare the architectural trade-offs behind different optimization strategies.
- We summarize emerging trends and open problems for scalable, efficient, and quality-aware RAG serving.

Rather than treating retrieval, caching, and decoding as separate topics, we emphasize the interactions among them. The resulting perspective is useful for researchers who want to understand the design space and for practitioners who need to choose an architecture under concrete service-level objectives.

2. Background: RAG Inference as a Systems Pipeline

An end-to-end RAG inference pipeline usually contains five stages:

- **Query encoding.** The input question is converted into one or more dense or sparse representations.
- **Vector retrieval.** The system searches a corpus or index for candidate knowledge items, typically through approximate nearest neighbor (ANN) search.
- **Reranking or filtering.** Retrieved items may be reranked, deduplicated, filtered, or fused before prompt construction.
- **Prompt construction.** The selected knowledge is packed into an LLM input prompt or long-context memory representation.
- **LLM decoding.** The model generates the answer while maintaining KV caches and other serving-side state.

Although this logical pipeline is simple, practical systems must handle several interacting bottlenecks.

2.1 Latency bottlenecks

Vector search can dominate latency when the corpus is large or when the index does not fit in fast memory. CPU/GPU transfers, SSD accesses, and synchronization across stages further increase response time. Tail latency can worsen when retrieval difficulty varies substantially between queries.

2.2 Memory bottlenecks

RAG serving must manage at least three types of memory pressure: vector indexes, retrieved context, and LLM KV caches. As corpora and model context windows grow, GPU memory alone becomes insufficient. This drives the use of CPU memory, external memory tiers, SmartSSDs, computational storage, and disaggregated architectures.

2.3 Throughput bottlenecks

A pipeline can become imbalanced when retrieval, reranking, and decoding have mismatched service rates. Systems that optimize only ANN speed or only decode speed often leave overall throughput limited by queueing and idle hardware.

2.4 Quality and freshness bottlenecks

Approximate retrieval, aggressive caching, and asynchronous updates can improve efficiency, but they may reduce recall or introduce stale evidence. Many recent works therefore explore quality-aware

adaptation instead of pure performance optimization.

2.5 Key metrics

A complete evaluation of RAG inference systems should include:

- average latency and tail latency,
- throughput under realistic concurrency,
- recall or answer quality,
- memory footprint,
- cost efficiency,
- update or freshness support, and
- energy efficiency where hardware specialization is involved.

These metrics motivate the taxonomy presented next.

3. Taxonomy of RAG Inference System Design

We organize the literature into five design axes: GPU acceleration, data reuse and cache-centric optimization, pipeline parallelism and scheduling, external-memory and storage-aware acceleration, and application-driven RAG systems. The categories are not mutually exclusive; the most effective systems increasingly combine ideas across them.

3.1 GPU acceleration

The first line of work adapts vector search and end-to-end RAG serving to GPU architectures. The main goal is to exploit the high parallelism and memory bandwidth of GPUs while reducing transfer overheads between host memory and accelerators.

Representative examples include **RAGO**, **Hermes**, **CAGRA**, **PilotANN**, **VecFlow**, and CPU/GPU collaborative filtering and re-ranking designs. At the algorithm level, GPU-oriented graph search and ANN structures aim to improve warp efficiency, reduce branch divergence, and sustain high memory throughput. At the system level, these works study how retrieval kernels, filtering, reranking, and LLM serving should be placed across devices.

Three recurring ideas appear in this category. First, GPU-native data structures can outperform CPU-origin designs when they are explicitly reorganized for SIMD-style execution and coalesced memory access. Second, hybrid CPU/GPU systems remain attractive when memory capacity is limited or when coarse filtering can be done cheaply before GPU reranking. Third, end-to-end overlap matters: a fast GPU retrieval kernel is most useful when the pipeline can hide data transfer and scheduling overhead rather than serializing every stage.

The key trade-off is that GPU-centric designs reduce latency and improve throughput, but they can increase system complexity and remain constrained by accelerator memory capacity. This limitation naturally connects the GPU-acceleration literature to the work on caching and external memory.

3.2 Data reuse and cache-centric optimization

The second design axis treats reuse as a primary systems strategy. Instead of redoing every retrieval and every model-side computation from scratch, these systems reuse chunks, knowledge states, reranker

results, or decoder-side caches.

Representative works include **Cache-Craft**, **CacheBlend**, **MemoRAG**, **HyperRAG**, **CacheFocus**, **METIS**, and closely related studies on approximate caching and cache-augmented generation. Although they differ in mechanism, they share a common observation: many workloads exhibit repetition in queries, documents, prompts, or intermediate states, and that redundancy can be exploited to reduce end-to-end cost.

This category spans several kinds of reuse:

- **Retrieved evidence reuse**, where popular or reusable chunks are cached.
- **Model-state reuse**, where previous computation helps serve related future queries.
- **KV-cache reuse**, where generation or reranking stages reuse decoder-side state.
- **Configuration reuse or adaptation**, where the system learns when to use smaller retrieval budgets or faster operating modes.

Cache-centric systems must carefully balance **hit rate**, **freshness**, and **storage cost**. A stale cache may save time but hurt correctness. A large cache can improve reuse but consume memory needed elsewhere. Some papers therefore move beyond static caching to dynamic repositioning, approximate matches, or quality-aware control policies. The broader message is that caching is no longer a small serving optimization; it is increasingly part of the core architecture of efficient RAG systems.

3.3 Pipeline parallelism and scheduling

A third line of work focuses on how retrieval and generation interact in time. Since RAG is a staged workflow, overall performance can improve dramatically when stages are pipelined rather than executed strictly in sequence.

Representative systems include **PipeRAG**, **RAGDoll**, **AquaPipe**, **Patchwork**, **ELERAG**, and **DGRAG**. These works investigate how retrieval, prompt assembly, and decoding can be overlapped across CPUs, GPUs, storage devices, or distributed nodes. Some emphasize adaptive pipeline partitioning, while others study offloading, lookahead retrieval, or edge-cloud cooperation.

This literature shows that two systems with similar component-level algorithms can exhibit very different end-to-end behavior depending on scheduling. Overlap can reduce apparent latency, increase hardware utilization, and smooth bursty workloads. However, pipelining also introduces coordination challenges:

- retrieval difficulty varies across queries,
- downstream stages may stall if upstream retrieval is slow or uncertain,
- speculative or lookahead retrieval can waste work if predictions are wrong, and
- load balancing becomes harder in heterogeneous environments.

The main contribution of this category is to shift attention from isolated stage optimization to service orchestration. In real deployments, queueing, overlap, and resource assignment often determine the final user-visible latency as much as ANN performance does.

3.4 External-memory and storage-aware acceleration

Large-scale RAG frequently exceeds the memory budget of a single accelerator or server. The fourth design axis therefore studies how to push retrieval closer to storage or how to extend the usable memory hierarchy beyond GPU HBM and CPU DRAM.

3.4.1 Processing in memory and near-data processing

Representative works in this subcategory include **Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs**, **NDSEARCH**, **Chameleon**, **REIS**, **DReX**, **DRIM-ANN**, and other computational-storage or accelerator-assisted designs. Their shared goal is to reduce data movement by performing more vector search work near where data resides.

The value proposition is clear: if vector indexes are large, moving them repeatedly to the host or accelerator is expensive in latency, bandwidth, and energy. Near-data designs instead offload parts of graph traversal, distance computation, or candidate filtering to SmartSSDs, computational storage, PIM/CIM platforms, or FPGA-based engines. Such approaches can improve scalability beyond GPU memory and sometimes deliver better energy efficiency.

The trade-off is hardware specialization. These systems can be highly effective for large retrieval-heavy deployments, but they may require non-standard platforms, custom runtimes, or redesign of indexing and scheduling logic.

3.4.2 Second-tier memory and disk-resident vector search

Another subcategory extends vector search into second-tier memory and disk-native indexing. The **DiskANN** family, **LM-DiskANN**, **FreshDiskANN**, **LSM-VEC**, **Starling**, **HM-ANN**, and recent SSD-aligned search designs illustrate the importance of I/O-aware ANN structures.

These systems address a central problem for RAG at scale: many corpora are too large for DRAM, let alone GPU memory. Disk-resident indexes therefore become essential. However, conventional ANN algorithms can degrade badly when random I/O dominates performance. The disk-native literature responds by redesigning graph layout, access ordering, update paths, and memory buffering to align search behavior with storage characteristics.

The main advantage of this line of work is scalability at relatively low memory cost. The main challenge is sensitivity to I/O patterns and update behavior. Still, this category is increasingly relevant because practical RAG deployments often need fresh, large, and economically manageable knowledge stores.

3.5 Application-driven RAG systems

The final category focuses on applications whose domain constraints reshape the system design. In the repository, examples include **SEFRQO** for query optimization and **AixelAsk** for table question answering.

Application-driven systems matter for two reasons. First, the retrieval object may differ substantially from open-domain text, for example structured tables, database hints, or enterprise knowledge artifacts. Second, service goals can change: a database assistant may prioritize plan quality and consistency, whereas a table QA system may need multi-step retrieval and reasoning over semi-structured evidence.

These workloads suggest that future RAG inference systems will not converge on a single universal architecture. Instead, different application domains will motivate different retrieval granularities, indexing strategies, update policies, prompt assembly methods, and caching mechanisms. The system community therefore needs abstractions that are general enough to support reuse, but flexible enough to capture workload-specific structure.

4. Cross-cutting Design Dimensions

Although the taxonomy is useful, many of the deepest insights come from dimensions that cut across categories.

4.1 Latency versus quality

Approximate retrieval, aggressive caching, and lookahead scheduling can reduce latency, but each may risk lower recall or reduced answer quality. Quality-aware systems such as METIS and AquaPipe are important because they treat this trade-off as explicit rather than accidental. A mature RAG serving stack should select operating points based on service objectives instead of assuming that the fastest path is always acceptable.

4.2 Throughput versus memory capacity

Throughput improvements often require more memory, whether for larger GPU-resident indexes, wider pipeline buffering, or larger caches. At the same time, keeping everything in fast memory is frequently impossible. This tension explains the rise of heterogeneous designs that combine GPUs, CPUs, CXL-like memory expansion, SSDs, and SmartSSDs.

4.3 Static indexing versus dynamic updates

Production knowledge bases change continuously. Yet many efficient ANN structures are easier to optimize when the index is relatively static. Works such as SPFresh, EdgeRAG, LSM-VEC, and FreshDiskANN point to a growing need for online indexing, incremental maintenance, and freshness-aware caching. Dynamic support will be essential for enterprise and real-time RAG workloads.

4.4 Monolithic versus heterogeneous architectures

Some systems aim for tightly integrated single-node performance, while others distribute work across CPUs, GPUs, storage devices, or edge-cloud environments. Heterogeneous architectures can scale further and use memory more efficiently, but they increase scheduling complexity and widen the design space. The right answer depends on the workload, deployment budget, and service-level objectives.

4.5 Retrieval-centric versus generation-centric optimization

The literature contains both retrieval-first and decoding-first optimizations. In practice, RAG quality and performance depend on the interaction between the two. Improving vector search alone is insufficient if prompt construction or decoding dominates latency. Likewise, aggressive LLM serving optimization may not help if retrieval remains the system bottleneck. The most promising direction is joint optimization across chunk selection, context packing, and decode scheduling.

5. Comparative View of Representative Systems

Table 1 summarizes representative systems from the repository and highlights the main targets and trade-offs visible from their positioning in the design space.

Year	Main target	Optimization level	Hardware setting	Main benefit	Key trade-off
2025	End-to-end RAG serving	system pipeline	GPU	lower latency and better throughput	implementation complexity
2025	large-scale efficient RAG	algorithm-system co-design	GPU	coordinated retrieval and serving efficiency	GPU-memory dependency
2024	GPU ANN search	index and kernel design	GPU	high parallel vector search	limited by accelerator capacity
2025	chunk-cache management	cache reuse	CPU/GPU serving	fewer repeated retrievals	freshness and cache size
2025	cached knowledge fusion	model-serving reuse	GPU/LLM serving	faster RAG serving	cache validity and freshness
2025	long-context processing	memory-enhanced retrieval	heterogeneous serving	better context reuse	added memory management

	Year	Main target	Optimization level	Hardware setting	Main benefit	Key trade-off
	2025	quality-efficiency trade-off	reranker KV-cache reuse	LLM serving stack	reduced repeated reranking cost	reuse depends on
	2025	quality-aware RAG	adaptive configuration	heterogeneous	better efficiency-quality control	controller complexi
	2025	adaptive pipelining	scheduling	heterogeneous	overlap retrieval and generation	balancing and back
	2025	lookahead retrieval	pipeline optimization	heterogeneous	reduced exposed retrieval latency	speculative waste r
	2025	unified RAG serving	framework design	heterogeneous	integrated end-to-end serving	broader orchestrati
	2025	edge-cloud RAG	distributed scheduling	edge plus cloud	scalable distributed retrieval	coordination overh
urch	2024	billion-scale retrieval	near-data processing	SmartSSD	scalability beyond host memory	hardware specializ
	2024	graph ANN acceleration	near-data traversal	storage-centric accelerator	lower data movement	platform dependen
	2025	retrieval-augmented LMs	disaggregated acceleration	heterogeneous accelerator system	memory and scalability gains	deployment comple
	2019	billion-point ANN	disk-native indexing	SSD	large-scale low-memory search	I/O-sensitive perfor
	2025	dynamic vector search	storage-aware updates	SSD plus memory	better update support at scale	design complexity
	2026	query optimization	application-specific RAG	database-oriented deployment	domain-aware retrieval benefits	narrower workload

The table highlights a broad trajectory in the literature. Early work often isolates a specific bottleneck such as GPU ANN search or disk-resident indexing, while more recent systems combine multiple techniques across caching, scheduling, and hardware hierarchy. This convergence suggests that future RAG platforms will increasingly resemble integrated data systems rather than loosely coupled model pipelines.

6. Research Trends

The repository literature suggests five major trends.

6.1 RAG is becoming a systems problem

The dominant shift is from model-centric improvement to end-to-end service design. The question is no longer only how to retrieve better evidence, but how to retrieve, assemble, and decode efficiently under strict service constraints.

6.2 Cache reuse is now a first-class design principle

Multiple recent papers center their contribution on chunk reuse, memory reuse, reranker reuse, or decoder-state reuse. This indicates a structural change in how the field thinks about RAG serving: reuse is becoming foundational rather than opportunistic.

6.3 Memory hierarchy is central

GPU memory alone is too small and expensive for many real deployments. Work on second-tier memory, storage-native indexes, CXL-style expansion, and computational storage shows that future RAG systems will need explicit memory-hierarchy management.

6.4 Near-data and storage-aware acceleration are becoming practical

SmartSSD, PIM, CIM, and storage-centric acceleration were once niche topics, but they now appear directly in RAG-serving discussions. This suggests that retrieval workloads are becoming large and repetitive enough for data-movement reduction to matter at system scale.

6.5 Quality-aware adaptation is emerging

Recent systems increasingly expose knobs that tune retrieval budgets, cache strategies, or pipeline behavior according to quality targets. This is a promising direction because production services rarely optimize only for raw speed.

7. Open Challenges and Future Directions

Despite rapid progress, several problems remain open.

7.1 Unified evaluation methodology

Different papers report different datasets, retrieval settings, hardware configurations, and quality metrics. This makes direct comparison difficult. A stronger benchmark culture for end-to-end RAG serving would help the community separate algorithmic gains from favorable evaluation assumptions.

7.2 Joint optimization of retrieval and decoding

Many current systems still optimize retrieval and generation separately. Future work should co-design chunk selection, prompt packing, decoding schedules, and cache reuse to maximize end-to-end utility instead of stage-local metrics.

7.3 Dynamic and continuously updated knowledge bases

Freshness is critical for practical RAG, but efficient support for online updates remains difficult. The challenge becomes harder when caches, ANN indexes, and storage-aware layouts must all remain consistent with rapidly changing corpora.

7.4 Long-context and memory-intensive RAG

As context windows grow, so do prompt assembly costs, memory traffic, and KV-cache pressure. Long-context RAG will require new combinations of compression, reuse, external memory, and hardware/software co-design.

7.5 Multi-tenant and production serving

Many published works optimize for single-workload scenarios, but real services must handle tenant isolation, workload bursts, fairness, and cost-aware scheduling. These concerns are underexplored relative to their practical importance.

7.6 Application-specific system design

Database assistants, enterprise QA, table reasoning, coding assistants, and edge deployments each impose different latency, accuracy, and update requirements. Future RAG systems will likely need modular architectures that can specialize per domain without losing the benefits of a shared systems substrate.

8. Conclusion

RAG-based LLM inference has rapidly evolved into a rich systems research area. The literature collected in this repository shows that efficient deployment depends on much more than retrieval accuracy. GPU-native vector search, cache-centric optimization, pipeline parallelism, external-memory support, and application-aware design all play central roles in determining latency, throughput, quality, and cost.

A useful way to read this landscape is through co-design. Retrieval and generation must be co-designed. Algorithms and schedulers must be co-designed. Memory hierarchies and serving policies must be co-designed. As RAG moves into larger, fresher, and more heterogeneous deployments, the strongest systems will be those that treat the end-to-end pipeline as an integrated architecture rather than a sequence of isolated components.

This survey draft, based on the repository paper list and outline, provides a foundation for that integrated view. It can be extended further with per-paper summaries, richer comparison tables, benchmark synthesis, and formal bibliography management, but even in its current form it makes one conclusion clear: the future of RAG depends as much on systems architecture as on model capability.